

# Midterm review!

## 1 Logic Levels

---

Assign  $V_{ih}$ ,  $V_{il}$ ,  $V_{oh}$ ,  $V_{ol}$  to maximize the noise margins.

Unity gain is where the slope is 1.

IL is the minimum input that would get a high, so that everything is below it.

IH is the maximum input needed before something is interpreted as a high, so that everything is above it.

We want to maximize noise margin, so we want to push the output signals OH OL as close to their maximum as possible, where as we want to have the widest area to have an input signal and want the IL IH in the middle.

Output low is the value that is at the boarder of our low, and output high is on the boarder of our output high. If we're inverting, than the low output would be the value associated with high input and vice versa.

## 2 Number Systems

---

There are three ways of interpreting a binary number.

Unsigned, sign/magnitude, or two's compliment.

$101_2$

Unsigned: 5

sign magnitude: -1

Two's compliment:  $-4 + 1 = -3$

-To make a negative, invert all the bits and add one

Going the other way:

$19_{10} =$

Unsigned:  $16+2+1 = 10011$

sign magnitude: 010011

Two's compliment: 010011

$-19_{10} =$

sign magnitude: 110011

Two's compliment: 101101

$37_{10}$

Hexadecimal: base 16,  $2 * 16 + 5 = 25_{16}$

Binary: 100101

### 3 CMOS Transistors

---

Series for AND, parallel for OR,

pMos pull low network

nMos pull low. To get the opposite, we have to add an inverter. For this, we connect the output to a not gate.

pMos is the one with a circle, pull to power.

EX: a 3 input OR gate.

We'll have 3 nmos connected to ground in parallel, so we'll have all 3 pmos connected to power in series. We then have to invert it.

The bits connected to ground (without the circle, aka nmos) are the logic you want. The bits connected to power (with the circle, aka pmos) are the compliment of the logic you want. Put them together, then invert the output.

### 4 Power Consumption

---

We have leakage current, and have to have a small resistor so that the leakage never gets to a active high level. We want to minimize power consumption, though, so want to make the resistor as big as possible to reduce the current draw. Thus, we have a hard max R constraint from the logic level, but want to maximize R.

$$P = P_{\text{dynamic}} + P_{\text{static}} = \alpha C V_{dd}^2 f + I_{\text{static}} V_{dd}$$

$\alpha$  is 1 if the clock is rising and falling every clock cycle,

0.5 if its only switching once per cycle (you only have to pay for charging, discharging is free)!

### 5 Combinational Logic Design

---

TRUTH TABLES

Output depends on current inputs only, no memory.

### 6 Finite State Machines

---

Synchronous sequential circuit, with a state transition diagram.

### 7 Timing

---

Flops. Two constraints are setup time and hold time.

Setup time: clock happens, signal has to arrive at least a setup time before the clock edge. clock skew: we need a longer clock period.

If we have two clocks, we don't want the output from 1 to change before clock 2 has passed its hold time.

We don't want our signal to take longer to propagate than the setup time before the clock with toggle.

$$\frac{P(\text{failure})}{\text{second}} = \left( \frac{T_0}{T_c} e^{-\frac{T_c - t_{\text{setup}}}{\tau}} \right)$$

Mean time between failures is  $\frac{1}{P \text{ failure / second}}$

## 8 Verilog

Write the idiom to get the hardware that we want. Clear and concise.

```
Combinational Logic with Boolean Eqns.
    assign y = (a & b) ^ (c | ~d);
Multiplexers
    assign y = s ? d1 : d0;
Comb logic with truth tables
    always_comb
        casez(in)
            3'b1xx: y <= 2'b11;
            3'b01x: y <= 2'b10;
            3'b001: y <= 2'b01;
            default: y <= 2'b00;
        endcase
```

Mealy machines are faster, moore are easier to analyze.

## 9 Arithmetic Circuits

Ripple Carry: simplest, carry signal just travels down.

Carry lookahead: We analyze things in terms of propagate or generate signals to make it faster

Parallel Prefix: reusing computations we've already done, each doubling adding only one more layer.